

Justice-Chain Sentinel Architecture Blueprint

Oct 1, 2026 · @siri

The recommended flow encrypts every clip on the device within seconds of capture, shares its key with the user's guardians, then sends the ciphertext through two paths at once: IPFS when online, the Nearby mesh when offline. A signed record of each incident is anchored on-chain, and the plaintext is deleted as soon as it is encrypted. The report explains each choice and the alternatives we rejected. It builds on the v1.3 handoff and the code audit of the ipfs-changes branch.

Design principles

Every design choice below is judged against three guarantees, in this order of priority:

1. **Never lose evidence.** A clip must survive the phone being smashed, taken or switched off during or right after an incident.
2. **Never leak evidence.** Nobody except the victim and their chosen guardians can watch a clip. That includes Pinata, IPFS nodes, the blockchain and anyone who takes the phone.
3. **Prove evidence is untampered.** A court can check that a clip is byte-for-byte what a specific device captured, and that it existed at a specific time.

Why this order: evidence that is lost cannot be proven, so availability comes first. Leaking comes second because a public victim video can cause harm that no court outcome repairs. Proof comes last because it only matters once the first two hold.

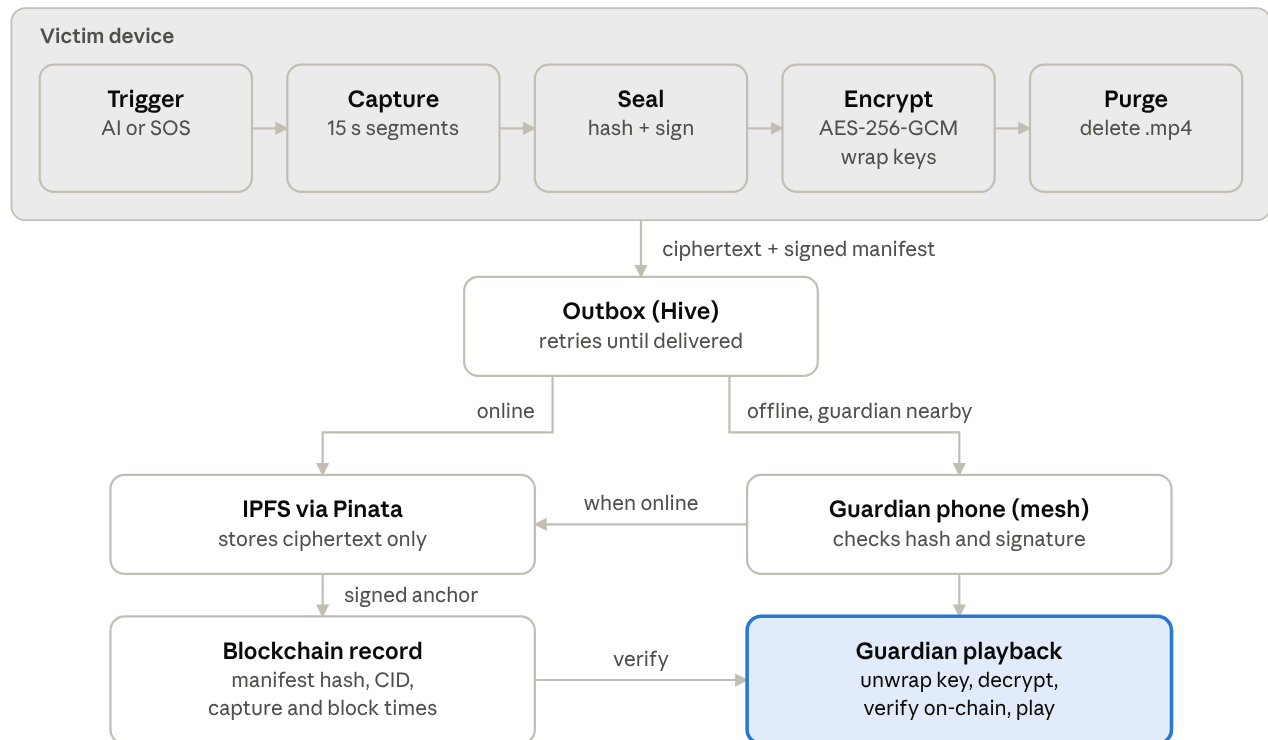
Why they rarely conflict in this design: the conflicts in the current code come from missing encryption. Once a clip is encrypted, it can be copied everywhere (guardians, IPFS, mesh) to meet guarantee 1 without breaking guarantee 2. Encryption is therefore the base the rest of the design depends on, not one feature among many.

Two engineering rules follow from the handoff and stay unchanged:

- **No server we operate.** Identity, keys and decisions stay on devices. Third-party infrastructure (Pinata, a blockchain) may store ciphertext and hashes but never holds a key.
- **No UI dependency in the emergency path.** Capture, sealing and upload must keep running if the screen is locked or the widget tree is disposed. Today the 45-second failsafe lives in `MainSafetyScreen`, which breaks this rule.

End-to-end flow

Plaintext never leaves the phone; two paths carry the ciphertext out



evidence flow · device to guardian and chain

Everything inside the shaded box happens on the victim's phone within seconds, and only ciphertext and a signed manifest leave it. The outbox sends each segment down whichever path works first. A guardian who got a copy over the mesh uploads and anchors it on the victim's behalf when they are next online. A guardian far away fetches the same ciphertext from IPFS instead.

Stage by stage

Each stage lists what we build, why, and what we considered and rejected. "Today" means the state of the ipfs-changes branch.

Stage	Today	Recommended	Why	Rejected alternative, and why not
1. Trigger	One TFLite window at 0.70 confidence fires recording	Fire on 2 of 3 consecutive windows at 0.80 or higher, plus a manual SOS button that bypasses the model	One noisy window (a shout at a football match) should not start a recording and an upload. A victim who can act must never depend on the model.	Cloud speech AI: needs a network, and sends raw audio off the device.
2. Capture	One 45-second .mp4 , controlled by the home screen widget	Three 15-second segments, run from <code>EmergencyController</code> inside an Android foreground service (iOS: background audio session, audio-only while locked)	If the phone is smashed at second 40, segments 1 and 2 are already sealed and leaving the device. A foreground service keeps the camera running when the screen locks.	Live streaming (WebRTC): needs a relay server and a network, and breaks both core rules.

Stage	Today	Recommended	Why	Rejected alternative, and why not
3. Seal	SHA-256 of the whole file, logged in Hive	SHA-256 per segment, each chained to the previous one, recorded in an incident manifest signed with the device's Ed25519 key	A hash alone proves a file has not changed. It does not prove which device made it. The Ed25519 key already exists but is never used to sign anything.	Hashing only the final file: it loses everything if capture is cut short.
4. Encrypt	None. Plaintext is uploaded to public IPFS	AES-256-GCM per segment, one random 256-bit key per incident, using the <code>cryptography</code> package already in <code>pubspec.yaml</code>	GCM is authenticated, so a flipped byte fails decryption loudly. It needs no new dependency. Fifteen-second segments (about 5–10 MB each) fit in memory, which the package's GCM API requires.	AES-256-CBC with the <code>encrypt</code> package, as in the handoff: CBC is unauthenticated, needs a separate HMAC to be safe, has padding-oracle history, and adds a dependency.

Stage	Today	Recommended	Why	Rejected alternative, and why not
5. Share the key	Not applicable	Wrap the incident key to the user's own X25519 key and split it into 2-of-3 shares for guardians, released only by victim approval or a check-in timeout (see Decisions)	Without this, destroying the phone destroys the only key and the evidence becomes unreadable forever.	Keeping the key only on the device: it fails guarantee 1 in the exact scenario the product exists for.
6. Purge plaintext	Plaintext <code>.mp4</code> is kept forever	Delete each plaintext segment right after it is encrypted and its key is wrapped. Keep the <code>.enc</code> only until the IPFS pin and chain anchor are confirmed, then delete it.	Plaintext then exists on the device for seconds, not until upload. That is much stronger than the handoff's "delete after upload" plan.	Deleting <code>.mp4</code> only after upload: an unlocked phone that is taken offline exposes every clip. "Secure overwrite": flash storage makes it unreliable, so encryption is the protection that actually works.
7. Distribute	Pinata upload only, fired once, no retry	An outbox in Hive. Each segment goes to IPFS when online and to any guardian in Nearby range, whichever happens first. Retry with backoff when connectivity returns.	During a blackout, the mesh is the only path. With a network, IPFS is the durable path. Running both paths means neither one is a single point of failure.	IPFS-only: fails during a blackout. Mesh-only: fails with no guardian nearby.

Stage	Today	Recommended	Why	Rejected alternative, and why not
8. Anchor	Contract exists; the app never calls it	One on-chain record per incident: manifest hash, manifest CID, capture time and node ID. Any party holding the signed manifest can submit it.	One transaction per incident, not per segment, keeps gas and complexity low. Anyone being able to submit it lets a guardian anchor for a victim whose phone is gone.	Anchoring every segment: three times the transactions for no extra proof, because the manifest already commits to every segment hash.
9. Retrieve and verify	Nothing built	A guardian screen: fetch the manifest, unwrap the key, decrypt, check every hash and the signature against the chain, then play	This is where the proof becomes useful. It is also the screen a lawyer or court would be shown.	An IPFS gateway link with no verification: it shows ciphertext, and proves nothing if it did not.

The incident manifest is the one data format that stages 3 to 9 all read and write. It is a small JSON file holding the incident ID, node ID, capture times, segment hashes and CIDs, the wrapped keys and the Ed25519 signature. We should fix its format first, because the mesh, the chain and the playback screen all consume it.

Keys and recovery

Each device holds three keypairs, one per job, all generated at registration and stored in `flutter_secure_storage`:

Key	Algorithm	Used for	Shared in the QR code
Identity	Ed25519 (exists today)	Signing incident manifests; the node ID is derived from it	Public key, as today
Encryption	X25519 (new)	Receiving wrapped incident keys from people who list this device as a guardian	Public key (new field)
Anchoring	secp256k1 (new)	Signing the on-chain anchor so the contract can check who made it with <code>ecrecover</code>	Address only

How a clip stays readable after the phone is destroyed: for each incident, the device makes one random AES key. It wraps that key once for its own X25519 key and once for each guardian's, using X25519 key agreement, then HKDF, then AES-GCM (a standard "sealed box"). The wrapped copies go into the manifest, which travels with the ciphertext to IPFS and to guardians. Any guardian can later open their copy with their own private key. Nobody else can, including Pinata and IPFS.

Why three keys, and not one:

- Using a single key for both signing and encryption is a known source of cross-protocol attacks. Converting Ed25519 keys to X25519 is possible, but the `cryptography` package does not expose it cleanly.
- Ethereum can only check secp256k1 signatures cheaply. Checking an Ed25519 signature on the EVM costs too much gas to be practical.

Why not Shamir secret sharing (k-of-n guardians) now: it protects against one rogue guardian, but it needs several guardians to cooperate before anyone can watch a clip. In an emergency, one reachable guardian is worth more than that protection. Revisit it once basic recovery works.

Migration: users registered before this change have no X25519 or secp256k1 key. On first launch after the update, we generate the missing keys and tell the user their guardians must re-scan the new QR code. Old guardian entries keep working for the mesh whitelist, but cannot receive keys until they re-pair.

PIN: hash it with Argon2id and a random salt (both are in the `cryptography` package), not plain SHA-256. A 6-digit PIN hashed with plain SHA-256 can be brute-forced in well under a second.

Offline mesh relay

The mesh is a one-hop handoff to a paired guardian who is physically nearby. Its job is to get ciphertext off the victim's phone before the phone is lost. When the guardian next has a network, their app uploads and anchors the evidence for the victim.

What it does:

1. On a distress trigger, the victim's phone advertises its node ID using Nearby Connections with `P2P_CLUSTER`.
2. A guardian's phone that is discovering sees the ID, checks it against the whitelist, and connects. The victim's phone checks the guardian's ID the same way.
3. The victim sends the signed manifest first, as a bytes payload, and then each `.enc` segment as a file payload.
4. The guardian rejects any file that has no manifest entry or whose hash does not match. It checks the manifest signature against the victim's Ed25519 public key saved at pairing.
5. The guardian sends back an acknowledgement. The victim's outbox marks the segment as delivered, which counts toward purging it.
6. When the guardian's phone is online, its own outbox uploads the segments to IPFS and submits the victim's signed anchor. The anchor is valid on-chain whoever submits it.

Fixes the current `mesh_service.dart` needs before any of this works:

- It reads the node ID from Hive key `node_id`, which nothing writes, so every device advertises as `UNKNOWN_NODE` and every whitelist check fails. It must use `IdentityService.getMyNodeId()`.
- If the file payload arrives before its metadata, the hash check is skipped and the file is accepted. The manifest-first order in step 3 closes this.
- It sends plaintext `.mp4` files. After this redesign, it sends only `.enc` segments.
- Nothing calls `startAdvertising` or `startDiscovery`, so the service never runs.
- On Android 11 and later, received files may arrive as a content URI, not a file path. We need to handle both.

What it deliberately does not do, and why not:

- **No multi-hop routing through strangers' phones.** `nearby_connections` has no routing layer. Relaying through untrusted devices would also expose who is in distress to strangers. Building both would take longer than the rest of this plan.

- **No always-on discovery on guardian phones.** Android limits background scanning, and continuous discovery drains the battery. Guardians run discovery while the app is open, and in a foreground service they can switch on (for example, when out together at night). We should tell users plainly that the mesh only works when a guardian is nearby and their app is listening.
- **No plaintext relay.** Guardians are trusted, but a guardian's phone can be lost or searched too.

Blockchain anchoring

The chain proves when an incident existed and which device signed it. It never stores video, keys or anything that identifies the victim by name.

Contract v2 (replaces `JusticeLedger.sol`). It keeps the interface from the handoff so Phase 2 can swap in behind it, with these changes:

Function	Change from today	Why
<code>storeEvidence(bytes32 manifestHash, string cid, uint64 capturedAt, bytes32 nodeId, bytes sig)</code>	Replaces <code>recordEvidence</code> . Uses <code>bytes32</code> instead of strings. Checks <code>sig</code> with <code>ecrecover</code> against the device's anchoring address. Rejects a hash that is already stored.	Anyone (a guardian, or a relay) can submit the record without being able to forge it. Duplicates would make verification ambiguous. <code>bytes32</code> costs less gas than <code>string</code> .
<code>verifyEvidence(bytes32 manifestHash) returns found, cid, capturedAt, anchoredAt, nodeId</code>	New. Looks up by hash.	Today's contract can only list records by victim ID. A court starts from a file, so it needs lookup by hash.
<code>getEvidenceHistory(bytes32 nodeId)</code>	Replaces <code>getEvidence</code> .	Same data, handoff naming.

Function	Change from today	Why
Stored times	Both <code>capturedAt</code> (device clock) and <code>anchoredAt</code> (<code>block.timestamp</code>)	The device clock can be wrong or set back on purpose. The block time cannot. Keeping both shows the gap honestly.

Who pays gas, and why not the obvious way. The simple approach is to put a funded Ethereum key inside the app. That works for a local Hardhat demo, where accounts are free, and it is how Phase 1 starts. It cannot ship, because anyone could pull the key out of the APK and drain it, and victims should not need crypto to be safe. The signed-anchor design above means a separate submitter can pay instead: a guardian's device, or later a small relayer. A relayer can delay a record but cannot forge or change it, so it does not break the no-trusted-server rule.

Phase path, and why Fabric waits:

1. **Local Hardhat** (next): contract v2 with tests, the app calling it through `web3dart` over RPC on the LAN.
2. **Public testnet** (Polygon Amoy is already in `hardhat.config.js`): anyone, including a court, can check a record on a public explorer without joining anything.
3. **Hyperledger Fabric**, only if a real police, lab or court partner commits to run nodes.

Fabric needs a certificate authority, orderers and peers for four organizations, plus a middleware API that we would have to host. That API breaks the no-server rule, and a consortium with no real members proves nothing to a court. Because contract v2 keeps the handoff's interface, moving to Fabric later means writing one adapter, not redesigning the app.

What we are not doing

These ideas came up in the handoff, in the code, or as obvious next steps. Each one is rejected or deferred for a stated reason.

Idea	Decision	Why not
AES-256-CBC via the <code>encrypt</code> package (handoff spec)	Rejected	It is unauthenticated and adds a dependency. AES-256-GCM from the package we already have is safer and simpler.
Delete the <code>.mp4</code> only after IPFS upload (handoff Priority 1)	Replaced	That leaves plaintext on the device for the whole upload window, which can be hours offline. We delete plaintext right after encryption instead.
Uploading plaintext to IPFS (current code)	Stop immediately	IPFS content is public and effectively permanent once others have fetched it. A leaked clip cannot be taken back.
Pinata JWT bundled in the APK as <code>.env</code> (current code)	Replace with a scoped, upload-only Pinata key	Anyone can unzip the APK and read the key. A scoped key limits the damage to uploads. Short-lived upload tokens would be better, but they need a server.
A funded Ethereum key inside the app	Local demo only	It can be extracted and drained, and victims should not need crypto. Signed anchors let someone else pay.
Hyperledger Fabric now (handoff Priority 5)	Deferred until a partner commits	Large operational cost, a hosted middleware API, and no real members to give it meaning. See Blockchain anchoring.
Multi-hop mesh through strangers	Rejected	No routing in <code>nearby_connections</code> , and it would expose victims to untrusted devices.
Shamir k-of-n key sharing	Deferred	Needs several guardians online to open one clip, which is wrong in an emergency. Revisit after basic recovery works.

Idea	Decision	Why not
Cloud AI for distress detection	Rejected	Needs a network and sends raw audio off the device.
Storing video or personal data on-chain	Rejected	Too expensive, and permanent. A victim could never delete it. The chain holds hashes and CIDs only.
Debug buttons in release builds ("Test IPFS", "Simulate AI Distress")	Hide behind <code>kDebugMode</code>	A stray tap could fire a false emergency upload, and they show an attacker what the app does.

Build order

The phases run in a straight line, because each one produces something the next depends on. Each phase ends with a check we can demo before moving on.

- 1. Foundation fixes.** Mesh node ID bug, Hive records addressed by key not index, the 45-second timer moved into `EmergencyController`, Argon2id PIN hashing, debug buttons hidden in release builds.
 - Done when: existing features behave the same, and the unit tests cover the vault and identity.
 - Why first: every later phase builds on these services, and the index bug would corrupt records written by later phases.
- 2. Encrypted evidence core.** X25519 and secp256k1 keys plus QR v2, `EvidenceEncryptor` (AES-256-GCM), key wrapping, the signed manifest, deleting plaintext after encryption, and the Hive outbox with retry and startup cleanup.
 - Done when: a recording produces only `.enc` files and a manifest, a second phone paired as guardian can decrypt it, and no `.mp4` is left on disk.
 - Why second: it stops the plaintext leak, and it fixes the manifest format that every later phase reads.
- 3. Anchoring on local Hardhat.** Contract v2 with Hardhat tests, deployed to a local node, and `web3dart` submitting the signed anchor after upload.
 - Done when: `verifyEvidence` on a real incident returns its CID and times.
 - Why here: it only needs a manifest hash and a CID, which phase 2 provides.

4. **Mesh relay.** Manifest-first transfer of `.enc` segments, delivery acknowledgements, and guardian upload-and-anchor on the victim's behalf.
 - Done when: with both phones in airplane mode plus Bluetooth and Wi-Fi on, a clip reaches the guardian, and appears on IPFS and on-chain once the guardian goes online.
 - Why after anchoring: the guardian's online step reuses the phase 3 code unchanged.
5. **Guardian playback and verification screen.** Fetch, unwrap, decrypt, check hashes, signature and chain record, then play.
 - Done when: a guardian can watch a clip and see "Verified: anchored in block N at time T".
6. **Capture hardening.** 15-second segments, an Android foreground service, and the 2-of-3 trigger rule.
 - Done when: locking the screen or killing the UI during recording still produces sealed segments.
 - Why this late: it is the hardest native work, and the pipeline has to be stable first so segments have somewhere to go.
7. **Public testnet.** Move from local Hardhat to Polygon Amoy with a guardian or relayer paying gas.
8. **Fabric**, only if a consortium partner commits. See Blockchain anchoring.

Why not the handoff's order (purge, mesh, playback, chain, Fabric): purging before encryption leaves IPFS as the only copy, in plaintext. Building the mesh before a fixed manifest format means building it twice.

Risks and open questions

Risks

Risk	Effect	Mitigation
A user registers with no guardians	Their key exists only on the phone, so evidence is lost if the phone is destroyed	Require at least one guardian before arming automatic recording, or show a clear warning

Risk	Effect	Mitigation
Camera gaps between 15-second segments	Up to about 1 second missed at each boundary with the <code>camera</code> plugin	Accept for now, and note the gap times in the manifest so they are not mistaken for tampering
The attacker sees the app recording	The victim may be put at more risk	A stealth recording mode with no "RECORDING" screen (later phase)
Pinata stops, deletes or rate-limits pins	The IPFS copy disappears	Guardians keep their mesh copies, and a second pinning service can be added behind the same outbox
Device clock is wrong	<code>capturedAt</code> is misleading	The chain's <code>anchoredAt</code> bounds it, and both times are shown in verification
Too many false triggers	Wasted uploads and gas, and users lose trust and turn the feature off	The 2-of-3 rule, and tracking the false-trigger rate in testing before raising sensitivity

Open questions for the team

- ✓ Should automatic recording be blocked until the user pairs at least one guardian?
- ✓ Is an Android-only target acceptable for now? The foreground service and Nearby behaviour differ a lot on iOS.
- ✓ Is a guardian allowed to watch every clip, or only after the victim or a timeout releases it? This changes whether keys are wrapped immediately or later.
- ✓ Do we have, or want, a police, lab or court contact? That decides whether Fabric is ever built.
- ✓ How long should `.enc` files stay on the device after confirmed upload, if at all, so the victim can review them offline?

Decisions (answers to the open questions)

All five open questions are now settled, and one new must-have is added: the app always listens in the background, with permission, and records only on a voice trigger or the SOS button.

Topic	Decision	What it changes in the design
Always-on listening (must-have)	With the user's explicit opt-in, a background listener runs all the time. Audio frames go to the on-device model in memory and are discarded; nothing is saved or sent until a voice trigger or the SOS button fires.	Android: a foreground service of type <code>microphone</code> (plus <code>camera</code> once recording), with a persistent notification, started while the app is visible and re-armed after reboot (newer Android versions may need one tap from a notification to restart the mic). iOS: a background audio session (<code>UIBackgroundModes: audio</code>) keeps the listener alive. A low-power keyword model keeps battery drain acceptable.
No guardian paired	Recording is never blocked. Capture, sealing, encryption and IPFS upload run with or without a guardian.	The incident key is always wrapped to the user's own key first. Guardian key shares are queued in the outbox and issued when a guardian is paired later, even mid-recording.
Android and iOS	Both platforms are required.	iOS cannot record video with the screen locked or the app in the background, so on iOS a background trigger records audio (plus location), and switches to video if the app is opened. Google's Nearby Connections is Android-only, so the mesh needs a Bluetooth LE transport that works between Android and iOS. Apple review needs a clear reason for background audio.

Topic	Decision	What it changes in the design
Guardian access	A guardian cannot watch a clip on their own. Recommended: split the incident key with Shamir secret sharing so any 2 of 3 holders are needed (guardians, plus optionally the victim's recovery key or an authority). A clip is released only when the victim approves it in the app, or after a check-in timeout (for example 72 hours with no response), and then only with a quorum.	One guardian alone (who might be the attacker) holds only ciphertext and one share. Guardians can still store and forward clips and submit the chain anchor, since neither needs the key.
Police, lab and court access at no token cost	Use Hyperledger Fabric. It is a permissioned ledger with no gas and no tokens; the only cost is running the nodes, which fit on a free-tier cloud VM or a college server during the pilot. Police, forensic lab and court each join as an organisation with their own peer.	Add a free public timestamp as a second proof: OpenTimestamps anchors a hash in Bitcoin at no cost to the user. Drop the Ethereum relay from the plan; keep Hardhat only for local demos. Authorities get access by the same key-share rule, never by holding every key.
<code>.enc</code> after upload	Delete the local <code>.enc</code> once the IPFS pin and the chain anchor are both confirmed.	The victim views later by re-fetching from IPFS and decrypting in the app with their own key (in Android Keystore / iOS Secure Enclave, behind biometrics plus PIN). Decryption happens in memory only, with screenshots blocked. A passphrase-protected key backup lets the victim view clips from a new phone if theirs is lost.